

ÉTUDE DE CAS

"UniStage" — Plateforme de gestion des offres de stage

Université de Labé — Licence 3 Informatique

Full-stack Angular & Spring Boot

1. Contexte et objectifs pédagogiques

Cette étude de cas conclut la formation "Développement Web" dispensée aux étudiants de troisième année de Licence en Informatique. Elle vise à mobiliser, dans un projet unique et cohérent, l'ensemble des compétences couvertes par les quatre grandes parties du programme : les fondamentaux du Web et le front-end Angular, la communication Angular / Back-end, le back-end Spring Boot et la base de données, ainsi que l'intégration front/back et les bonnes pratiques.

À l'issue de ce travail, chaque groupe devra être en mesure de concevoir, développer et présenter une application Web full-stack fonctionnelle, sécurisée et correctement structurée.

Objectifs spécifiques

- Concevoir une architecture client/serveur avec une API REST documentée
- Développer une interface Angular modulaire (composants, services, routing, guards)
- Implémenter un CRUD complet côté back-end avec Spring Boot et Spring Data JPA
- Sécuriser l'application avec Spring Security et une authentification JWT
- Gérer les échanges front/back via HttpClient, y compris la gestion des erreurs et le CORS

2. Présentation du cas d'étude

L'Université de Labé souhaite se doter d'une plateforme numérique, "UniStage", permettant de mettre en relation les étudiants finissants avec des entreprises proposant des offres de stage. La plateforme doit permettre la publication d'offres, le dépôt de candidatures et le suivi de leur traitement, avec un espace dédié pour chaque type d'utilisateur.

Ce scénario a été choisi car il couvre naturellement des entités liées (utilisateurs, offres, candidatures), plusieurs rôles avec des droits différenciés, et un cycle de vie de données représentatif d'une application métier réelle.

3. Acteurs du système

- Étudiant : consulte les offres, dépose des candidatures, suit leur statut
- Entreprise : publie des offres de stage, consulte et traite les candidatures reçues

- Administrateur : valide les comptes entreprises, supervise les offres et modère la plateforme
- Visiteur (non authentifié) : consulte la liste publique des offres de stage

4. Spécifications fonctionnelles par module

Les fonctionnalités sont regroupées selon les quatre parties du programme, afin que chaque étudiant puisse relier directement son travail aux notions vues en formation.

Etape 1 — Fondamentaux du Web et front-end Angular

1. Page d'accueil publique listant les offres de stage ouvertes (HTML sémantique, mise en page responsive avec Flexbox/CSS Grid, Media Queries)
2. Formulaire d'inscription et de connexion (Étudiant / Entreprise) avec formulaire réactif (Reactive Forms) et validations (champs requis, format email, longueur du mot de passe)
3. Structuration de l'application en composants réutilisables (carte d'offre, barre de navigation, formulaire de candidature) avec communication parent-enfant (@Input / @Output)
4. Utilisation du nouveau Control Flow (@if, @for, @switch) pour l'affichage conditionnel et les listes d'offres

Etape 2 — Angular et communication avec le back-end

5. Création de services Angular avec injection de dépendances pour chaque ressource (offres, candidatures, utilisateurs)
6. Consommation de l'API REST via HttpClient : lecture des offres, soumission d'une candidature, mise à jour de statut
7. Gestion centralisée des erreurs HTTP (401, 403, 404, 500) avec messages adaptés à l'utilisateur
8. Mise en place du routing avec au moins trois routes protégées par un guard (ex. : espace étudiant, espace entreprise, espace admin) selon le rôle

Etape 3 — Back-end avec Spring Boot et base de données

9. Modélisation des entités JPA (Utilisateur, Etudiant, Entreprise, OffreStage, Candidature) et de leurs relations (voir modèle de données §5)

10. Mise en place des Repository Spring Data JPA avec au moins deux Query Methods personnalisées (ex. : recherche d'offres par secteur, candidatures par statut)
11. Développement des contrôleurs REST exposant les opérations CRUD nécessaires, avec DTOs pour ne pas exposer directement les entités
12. Gestion des codes de statut HTTP appropriés pour chaque opération (200, 201, 400, 404, etc.)

Etape 4 — Intégration front/back et bonnes pratiques

13. Authentification par JWT : génération du jeton à la connexion, vérification à chaque requête protégée
14. Protection des endpoints selon le rôle (ETUDIANT, ENTREPRISE, ADMIN) via Spring Security
15. Configuration correcte du CORS entre l'application Angular et l'API Spring Boot
16. Stockage sécurisé du token côté client et déconnexion effective
17. Organisation du code, gestion de version avec Git (commits réguliers et explicites), fichier README expliquant l'installation et l'exécution du projet

5. Modèle de données suggéré

Ce modèle est indicatif : les groupes peuvent l'adapter tant que la cohérence des relations et des contraintes métier est respectée.

Entité	Attributs principaux	Relations
Utilisateur	id, nom, prenom, email, motDePasse (hashé), role (ETUDIANT / ENTREPRISE / ADMIN)	1..1 avec Etudiant ou Entreprise selon le rôle
Etudiant	id, matricule, filiere, niveau, telephone, cv (fichier ou lien)	1..N Candidature
Entreprise	id, raisonSociale, secteurActivite, adresse, siteWeb, statutValidation	1..N OffreStage
OffreStage	id, titre, description, competencesRequises, dateDebut, dateFin, lieu, statut (OUVERTE/FERMEE)	N..1 Entreprise · 1..N Candidature

Candidature	id, dateSoumission, lettreMotivation, statut (EN_ATTENTE/ACCEPTEE/REJETEE)	N..1 Etudiant · N..1 OffreStage
-------------	---	------------------------------------

6. Livrables attendus

- Dépôt Git contenant le code source front-end et back-end, avec historique de commits
- Fichier README décrivant l'architecture, les choix techniques et les instructions d'installation
- Collection Postman (ou export équivalent) des principaux appels API